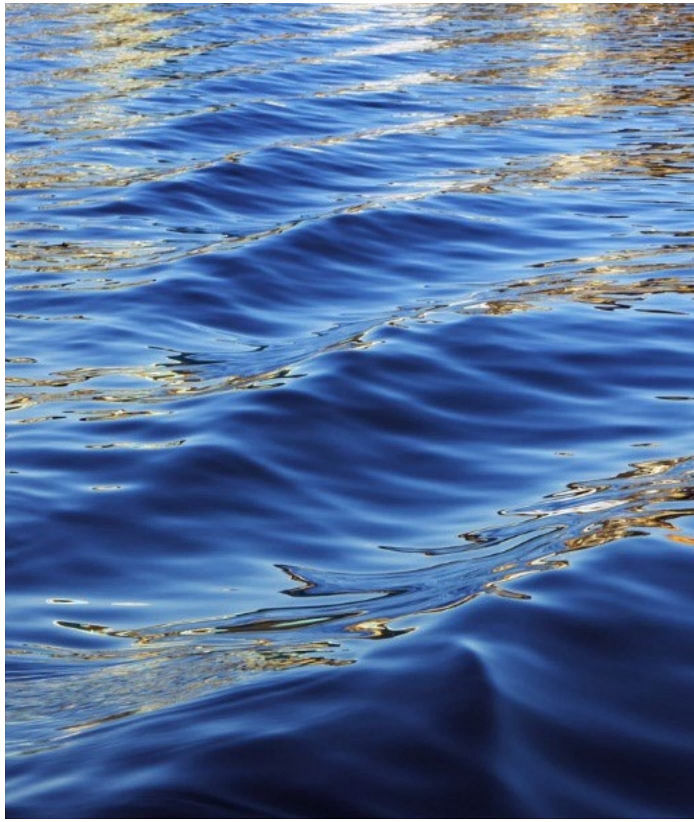
Optimal Milk-Run Routing

Miguel A. López-Carmona
Alvaro Paricio-García



EUGLOH
European University Alliance for Global Health

Co-funded by the European Union

Universidad de Alcalá

Learning objectives

Learning objectives

- Understand the optimal routes design problem.
- Formulate as a LP problem, and understand complexity.
- Understand how heuristic algorithms can help:
  - Algorithm Clarke-Wright.

The vehicle routing problem (VRP)

EUGLOH

- **Find minimum cost routes from a single origin to multiple destinations using multiple vehicles.**
- Who needs to solve the problem?
  - **Carriers: retailers, distributors, manufacturers** ←el last mile problem
  - **Carriers** - LTL, parcel
  - **Service companies**: repair, waste, utilities, post office, removal of items.
- Types of problems
  - **Commercial delivery** (retailers, distributors, manufacturers)
  - **Commercial collection** (retailers, distributors, manufacturers)
  - **Mixed pickup and delivery** (LTL and parcel carriers)
  - **Residential Appointment** (online grocery, medical gases, repair)
  - **Residential sweeping** (mail, waste, utilities, removal of items, etc.)

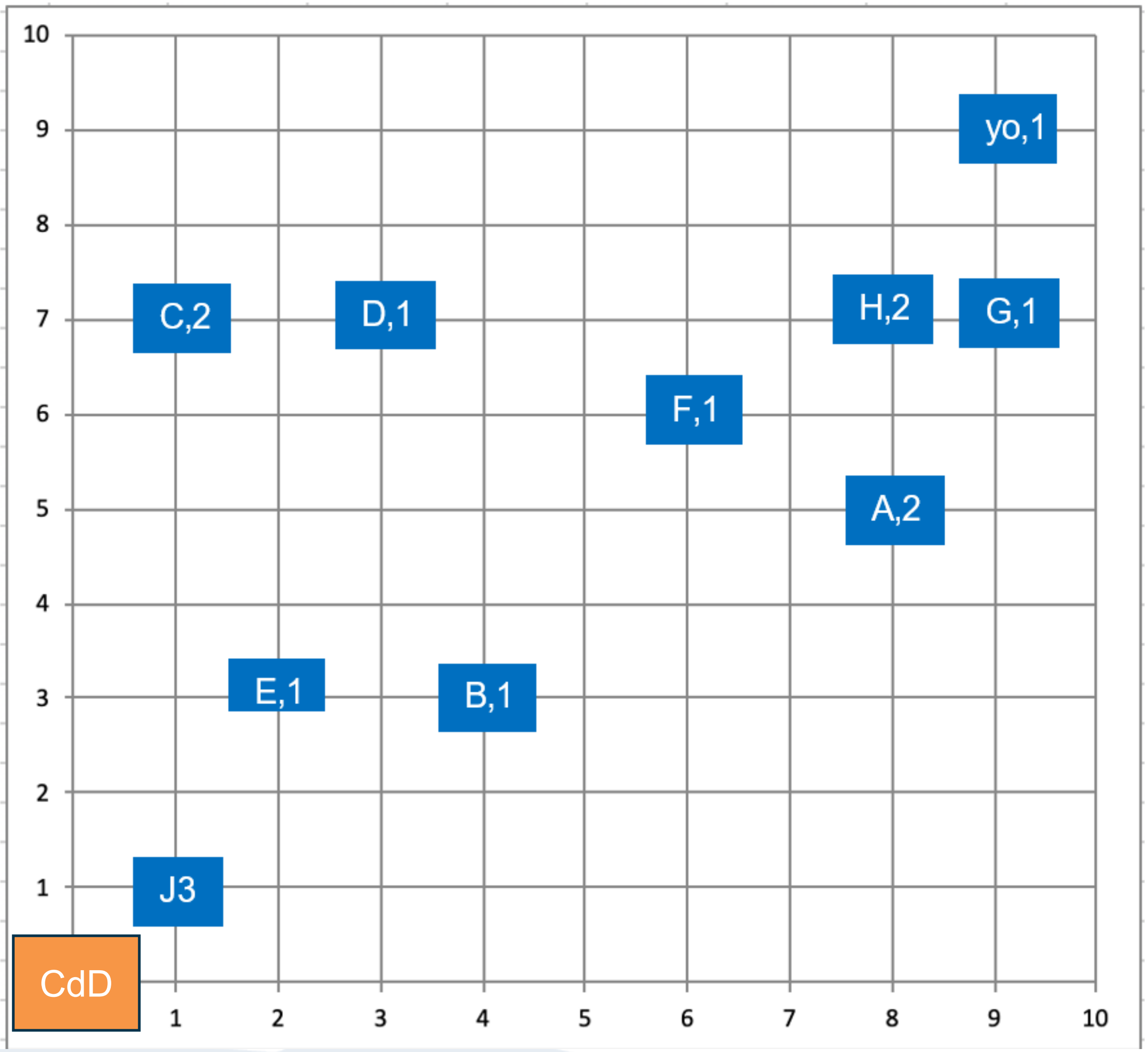
Example: "last mile" delivery of ice on request

- Your company manufactures and delivers environmentally friendly sustainable ice to very demanding customers throughout the city. They require the ice to be manufactured with high control and delivered very quickly.
- Orders are received throughout the day, with a half-hour delivery commitment. If an order arrives at, say, 1:25 or 1:00 p.m., it will be included in the 1:30 p.m. deliveries. All deliveries are made from a central ice manufacturing plant.

- You only have four electric delivery vehicles that can hold 5 bags of ice each. In addition, a trip cannot exceed 30 minutes to be ready for the next deliveries.

- Every 30 minutes all outstanding orders are collected and the optimal route for the four vehicles must be determined very quickly. Any delivery not made within half an hour incurs a very high penalty.

Customized ice delivery: orders from 1:30 p.m. onwards



Décimas de kilómetro

Details:

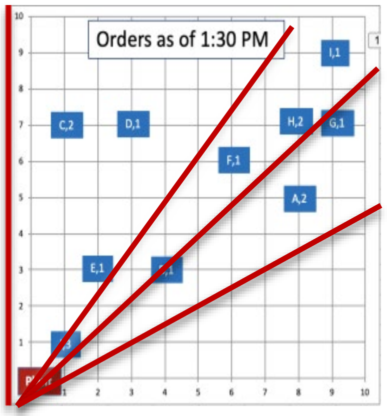
- All orders are delivered in electric vehicles and must start and end at the CD.
- Ten orders (A - J) were received for a total of 15 bags of ice.
- Each electric vehicle can travel at an average of 12 kilometers per hour.
- The distances on the map are 0.1 kilometers.
- Each delivery takes 5 minutes because it is loaded into your customized refrigerator.
- Each electric vehicle can hold up to 5 bags of ice.
- The routes may not exceed 30 minutes.

Let's solve this with Clark-Wright

Solution approaches

- Many approaches available
- Basically, it is a balance between speed and quality of the solution.
  - Simple heuristics
    - Fast but quality of solution may vary
    - Example: sweep method
  - Optimization
    - Find an optimum, but it may take a long time.
    - Example: Mixed integer linear programming (MILP)
  - Advanced heuristics
    - Not as fast as the simple ones, but generally has a better solution quality.
    - Example: Clark-Wright saving algorithm

Sweep heuristic:



Linear programming (MILP)

$$
\begin{aligned}
& Min \sum_j C_j Y_j \\
& s.t. \\
& \sum_{j=1}^{J} a_{ij} Y_j \geq D_i \ \forall i \\
& \sum_{j=1}^{J} Y_j \leq V \\
& Y_j = \{0,1\} \ \forall j
\end{aligned}
$$

Vehicle Routing Optimization Model

- Minimize distance or maximize demand with a LP-Solver

LP Problem Statement

- Design up to R delivery routes starting and ending at a Central Deport, so that:
  - All the customers are served at most once in the demand period.
  - Optimality criteria is applied (or'd):
    - Minimize total distance.
    - Maximize total demand.
  - And feasibility and business constraints are applied:
    - Each route must not exceed a maximum distance D.
    - Each route should not exceed more than T hours.
    - Each vehicle, can only deliver certain payload P.
    - Others…

LP Model - Model Inputs

- Central depot: DC
- Set of N nodes (customer locations) with GPS coordinates and demand
  - Demands of the nodes, Q={q_j }
  - Distance matrix between nodes, DM={d_{ij}}
- Mean vehicle speed S
  - Traveltime matrix, TM={t_{ij}}
- Parameters:
  - Max number of routes R to find → bounded by N
  - Route max distance (D)
  - Route max travel time (T)
  - Vehicle max payload (P)
  - Other constraints…

LP Model - Decision Variables

- Our objective is to find:
  a) How many routes are required to optimally serve the nodes.
     - The problem should detect the R optimal routes (R<=N, upper bound).

  b) Which nodes are covered by each route:
     - $y_j^r \in \{0,1\}$: if node j is visited in route r → 1, else 0

  c) The node connections in the routes (travel order):
     - $x_{ij}^r \in \{0,1\}$: if route r includes arc from node i to j → 1, else 0

LP Model - Objective Function

- Option 1: Minimize total distance
  - Minimize $\sum\sum\sum d_{ij} * x_{ij}^{r}$
- Option 2: Maximize total demand served
  - Maximize $\sum\sum q_{j} * y_{j}^{r}$

LP Model - Constraints

1. Route starts/ends at DC: 1 inbound, 1 outbound per route
2. Each customer visited at most once: $\sum y_j^r \leq 1$
3. Distance limit per route: $\sum d_{ij} * x_{ij}^r \leq$ D
4. Flow balance: In = Out = y_j^r for each node j
5. Optional: Subtour elimination

Lp Solver Implementation (Excel)

- Columns: Route, From, To, Distance, Demand, x_{ij}^{r}
- Contributions column: = x_{ij}^{r} * Distance or = x_{ij}^{r} * Demand
- Objective cell: SUM(Contributions)
- Solver: Maximize or Minimize objective by changing x_{ij}^{r}
- Additional constraints as per model

Complexity…

- Complexity of the LP problem (number of variables) is N^3:
  + R*N*(N-1) possible node connections
  + R*N nodes coverage in the routes
- If additional DC's → N^3 *DC

- Example:
  - Our distribution problem contains one DC and 14 sites → LP problem with aprox. 2.700 variables to consider.

- Heuristics can help!!!
  - Clarke-Wright algorithm.

Vehicle Routing Optimization Heuristics

- Clarke-Wright Algorithm

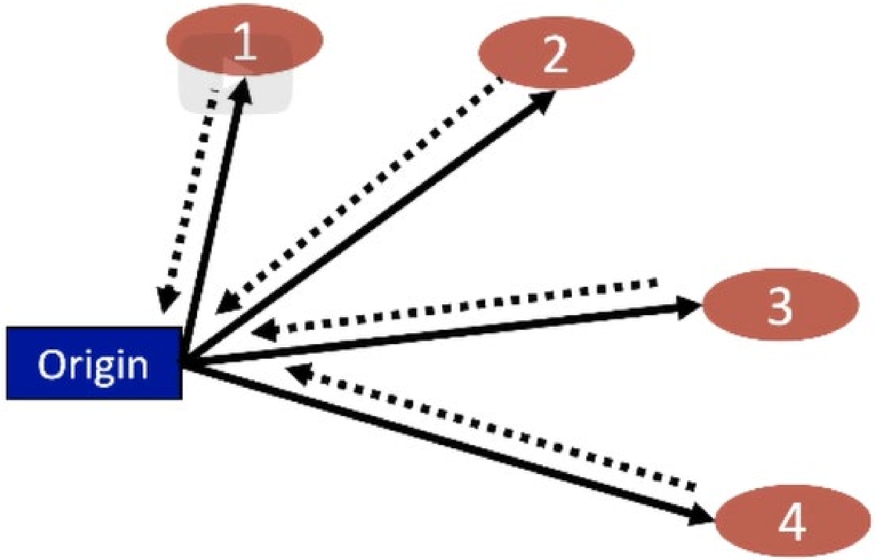
Clarke - Wright Algorithm

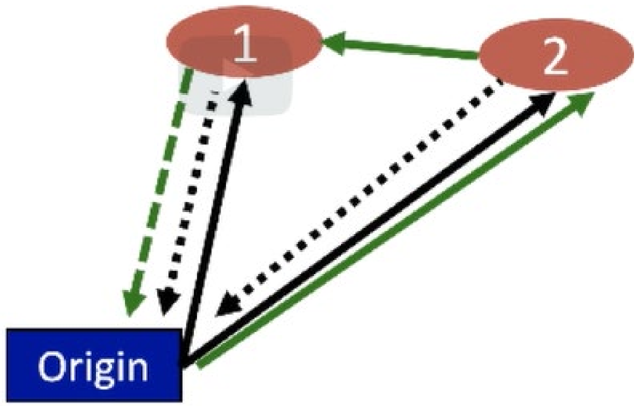
- (1) Start with a complete solution (round trip) to each node.
  - Each trip is loaded and returns empty.
- (2) We search if adding any node to the trip can save cost:
  - We studied whether putting node 2 in the first trip could save us some money:

$$2c_{O1} + 2c_{O2} > c_{O1} + c_{12} + c_{2O}$$

$$c_{O1} + c_{2O} - c_{12} > 0$$

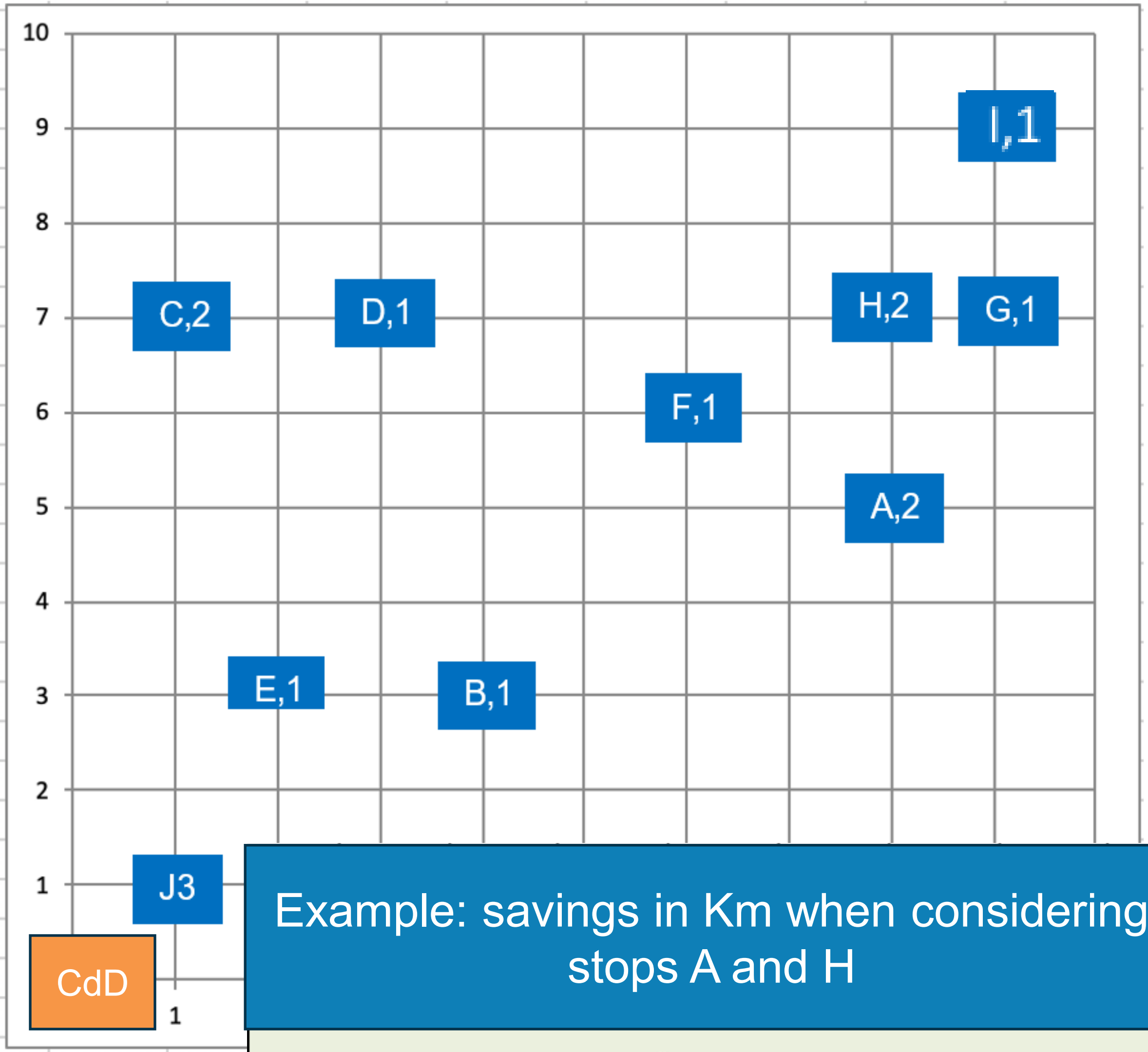
- (3) The operation is repeated for all possible tuples of nodes, selecting the lowest cost options.
- (4) We always maintain capacity restrictions.
- (5) The "inner path" nodes cannot be added: they must be at the end.





Solution

1. Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$, j) of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under considerati $s_{i,j}$, include arc (i, j) in a path only if:
   - No route $s_{i,j}$r vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.



Example: savings in Km when considering stops A and H

$$s_{A,H} = c_{DC-A} + c_{DC-H} - c_{H-A}$$

$$= 0{,}94 + 1{,}06 - 0{,}20 = 1{,}74$$

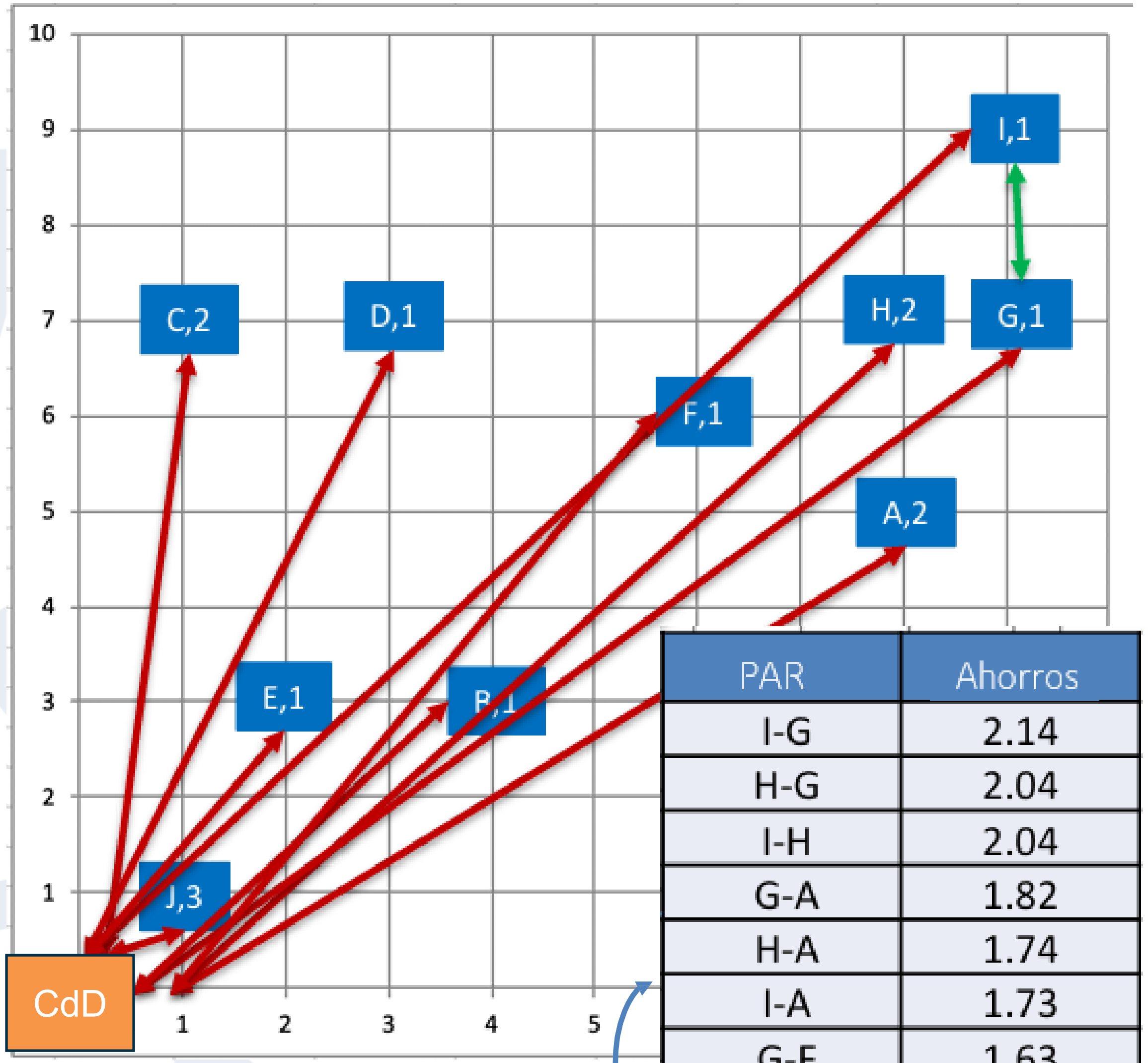
Distances between sites

Dist	A	B	C	D	E	F	G	H	I	J
A	-									
B	0.45	-								
C	0.73	0.50	-							
D	0.54	0.41	0.20	-						
E	0.63	0.20	0.41	0.41	-					
F	0.22	0.36	0.51	0.32	0.50	-				
G	0.22	0.64	0.80	0.60	0.81	0.32	-			
H	0.20	0.57	0.70	0.50	0.72	0.22	0.10	-		
I	0.41	0.78	0.82	0.63	0.92	0.42	0.20	0.22	-	
J	0.81	0.36	0.60	0.63	0.22	0.71	1.00	0.92	1.13	-
DC	0.94	0.50	0.71	0.76	0.36	0.85	1.14	1.06	1.27	0.14

Savings (s_{ij})

Savings	A	B	C	D	E	F	G	H	I	J
A	-									
B	1.00									
C	0.92	0.70								
D	1.10	0.79	1.21							
E	0.61	0.60	0.59	0.65						
F	1.52	0.94	1.00	1.25	0.66					
G	1.82	0.96	1.01	1.26	0.65	1.63				
H	1.74	0.93	1.01	1.26	0.64	1.62	2.04			
I	1.73	0.92	1.08	1.33	0.64	1.62	2.14	2.04		
J	0.24	0.24	0.21	0.23	0.24	0.24	0.24	0.24	0.24	

Bidding solution



Next 9 combinations

PAR	Ahorros
I-G	2.14
H-G	2.04
I-H	2.04
G-A	1.82
H-A	1.74
I-A	1.73
G-F	1.63
H-F	1.62
I-F	1.62

1. Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$, j) of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under consideratis $s_{i,j}$, include arc (i, j) in a path only if:
   - No route $s_{i,j}$r vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.

Total initial distance (outward) = 15,48 km

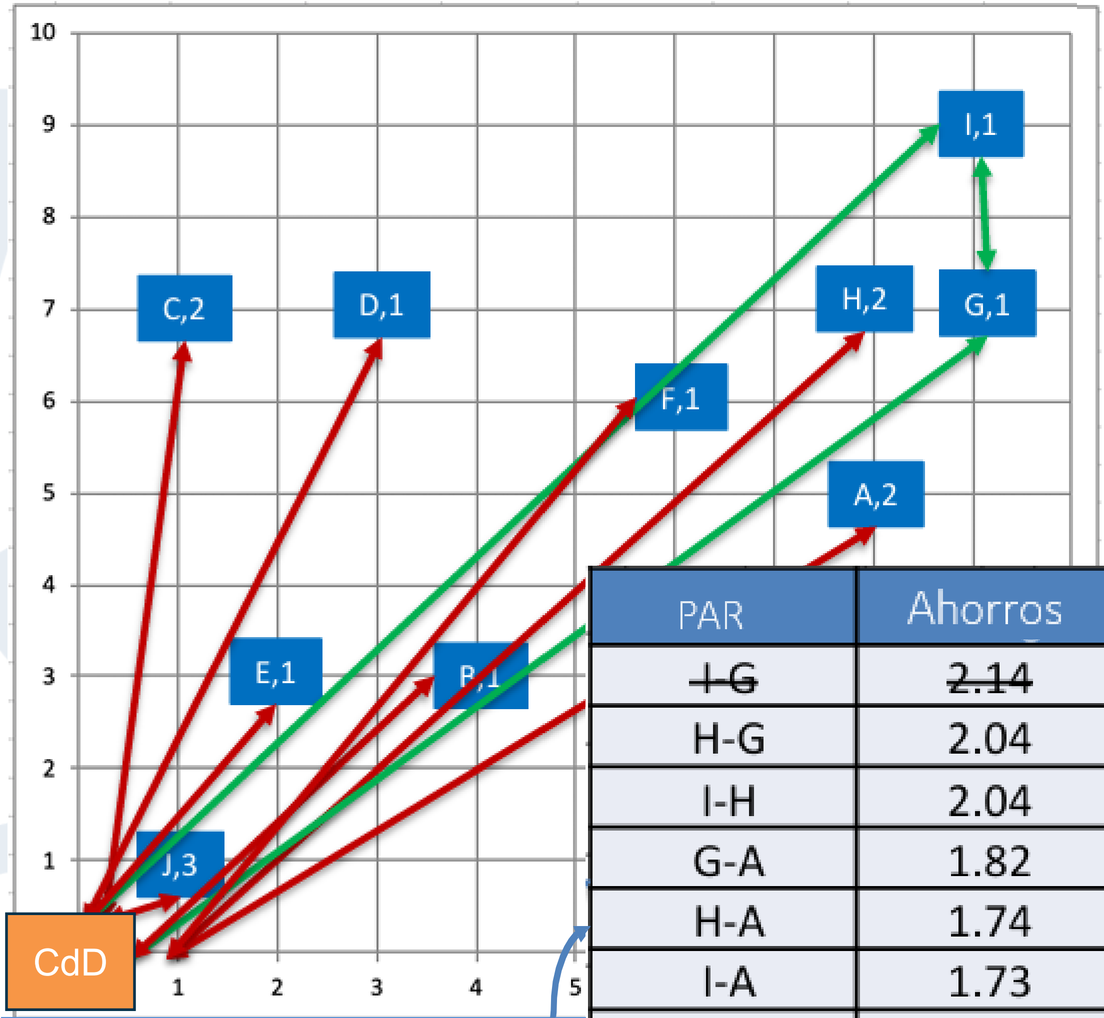
Consider joining I-G

capacity = 1 bag + 1 bag = 2 ≤ 5 SI

time = (2.61 km)/(0.2 kpm) + 2(5 En) = 23.05 min. SI

Total distance = 15,48 - 2,14 = 13,34 km

Bidding solution



PAR	Ahorros
~~I-G~~	~~2.14~~
H-G	2.04
I-H	2.04
G-A	1.82
H-A	1.74
I-A	1.73
G-F	1.63
H-F	1.62
I-F	1.62

Next 9 combinations

1. Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$, j) of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under consideratis $s_{i,j}$, include arc (i, j) in a path only if:
   - No route $s_{i,j}$r vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.

Total initial distance (outward) = 15,48 km

Consider joining I-G

capacity = 1 bag + 1 bag = 2 ≤ 5 SI

time = (2.61 km)/(0.2 kpm) + 2(5 En) = 23.05 min. SI

Total distance = 15.48 - 2.14 = 13.34 km

Consider joining HG

capacity = 2 bags + 2 bags = 4 ≤ 5 SI

time = (2.63 km)/(0.2 kpm) + 3(5 En) = 28.15 min. SI

Total distance = 13.34 - 2.04 = 11.30 km

Bidding solution

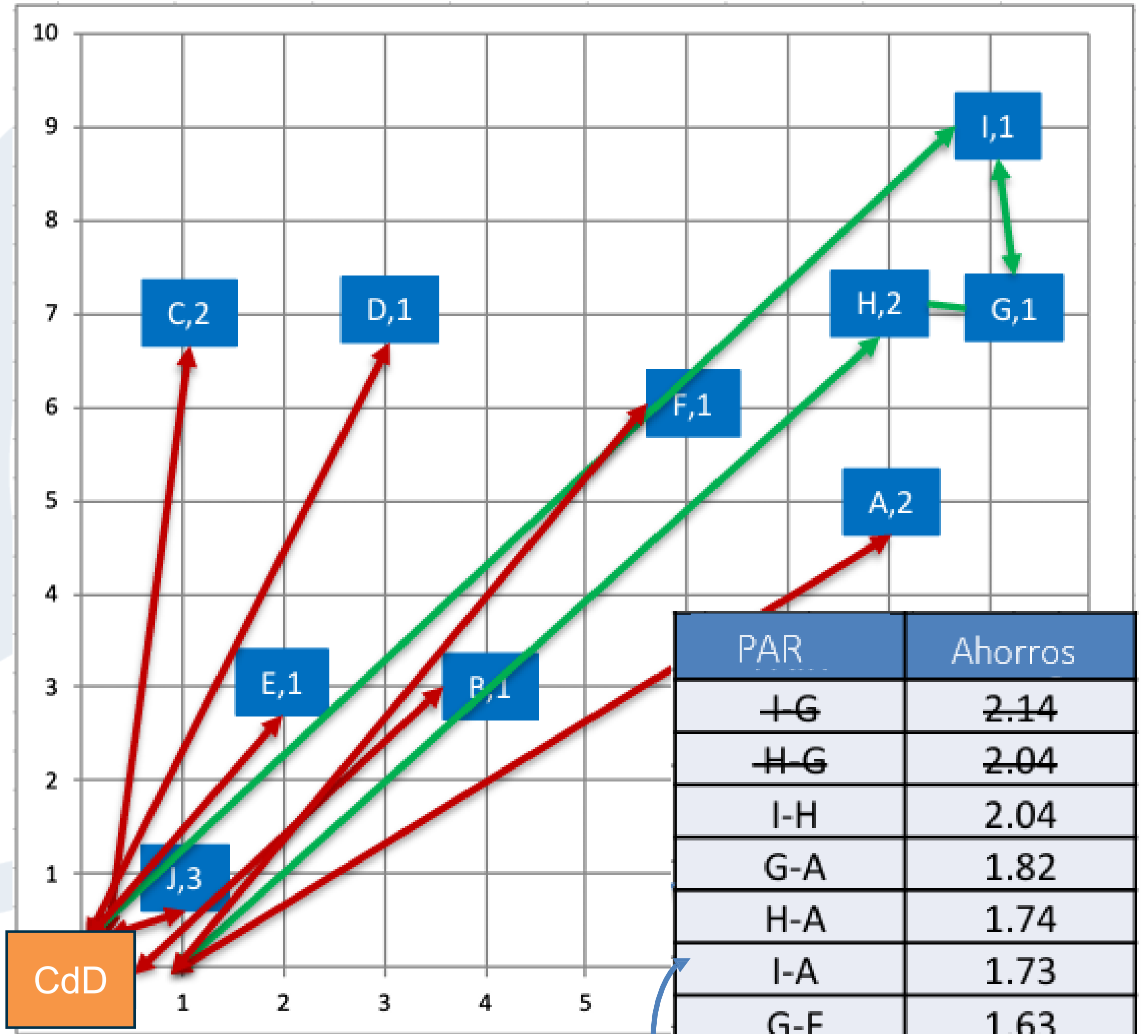
1.Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$, j) of demand nodes.

2.Sort and process savings in descending order of magnitude.

3.For the savings under considerati $s_{i,j}$, include arc (i, j) in a path only if:

- No route $s_{i,j}$r vehicle restrictions will be violated by adding it to a route, and
- Nodes i and j are the first or last nodes to/from the origin on your current route.

4.If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.



PAR	Ahorros
~~I-G~~	~~2.14~~
~~H-G~~	~~2.04~~
I-H	2.04
G-A	1.82
H-A	1.74
I-A	1.73
G-F	1.63
H-F	1.62
I-F	1.62

Next 9 combinations

Total initial distance (outward) = 15,48 km

Consider joining I-G

capacity = 1 bag + 1 bag = 2 ≤ 5 SI

time = (2.61 km)/(0.2 kpm) + 2(5 En) = 23.05 min. SI

Total distance = 15,48 - 2,14 = 13,34 km

Consider joining HG

capacity = 2 bags + 2 bags = 4 ≤ 5 SI

time = (2.63 km)/(0.2 kpm) + 3(5 En) = 28.15 min. SI

Total distance = 13.34 - 2.04 = 11.30 km

Consider joining IH,GA, HA, IA, IA, GF, HF NO

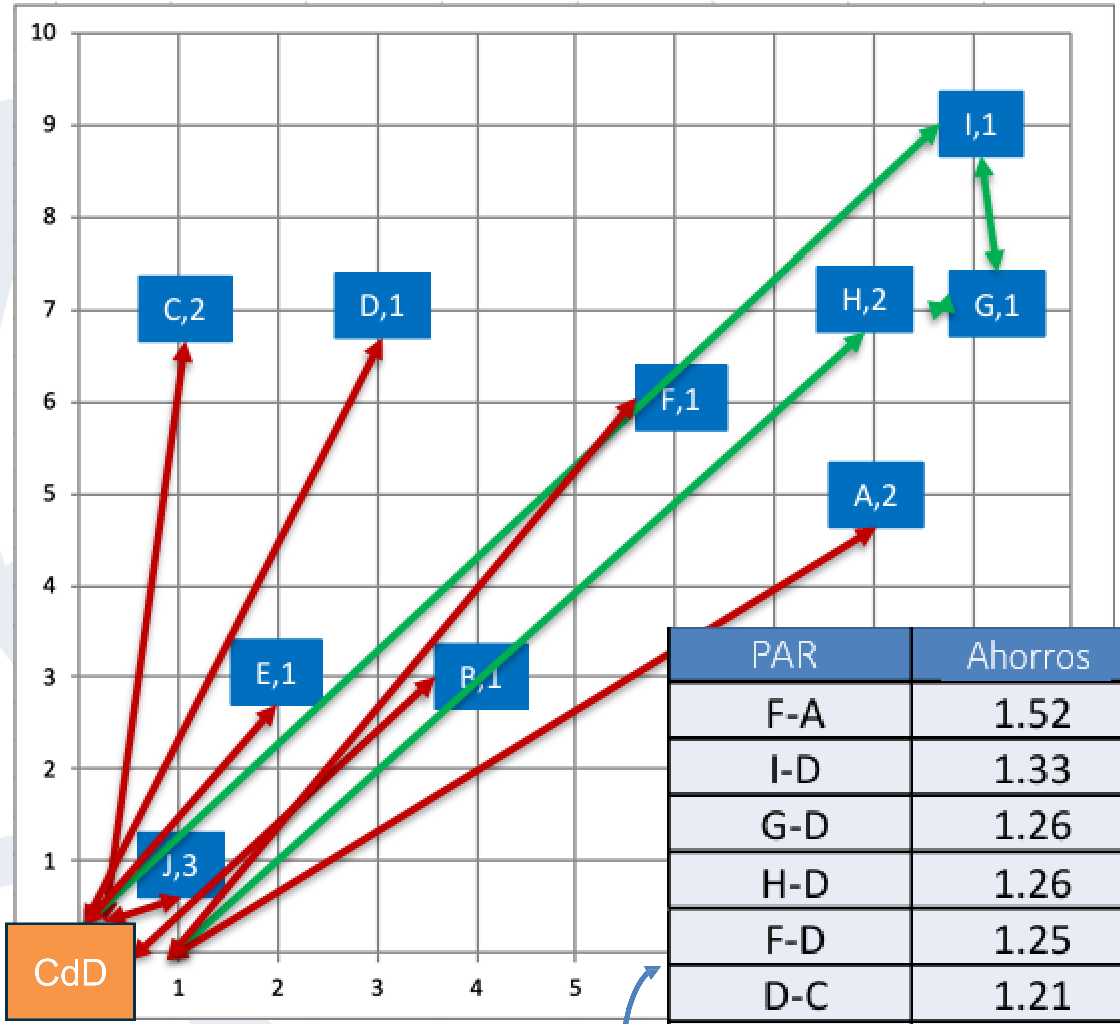
Consider joining IF

capacity = 4 bags + 1 bag = 5 ≤ 5 SI

time = (2.64 km)/(0.2 kpm) + 4(5 En) = 33.20 min NO

Bidding solution

1. Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$ of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under consideration $s_{i,j}$, include arc (i, j) in a path only if:
   - No route or vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.



Next 9 combinations

PAR	Ahorros
F-A	1.52
I-D	1.33
G-D	1.26
H-D	1.26
F-D	1.25
D-C	1.21
D-A	1.10
I-C	1.08
G-C	1.01

Current total distance of the route = 11.30 km

Consider joining F-A

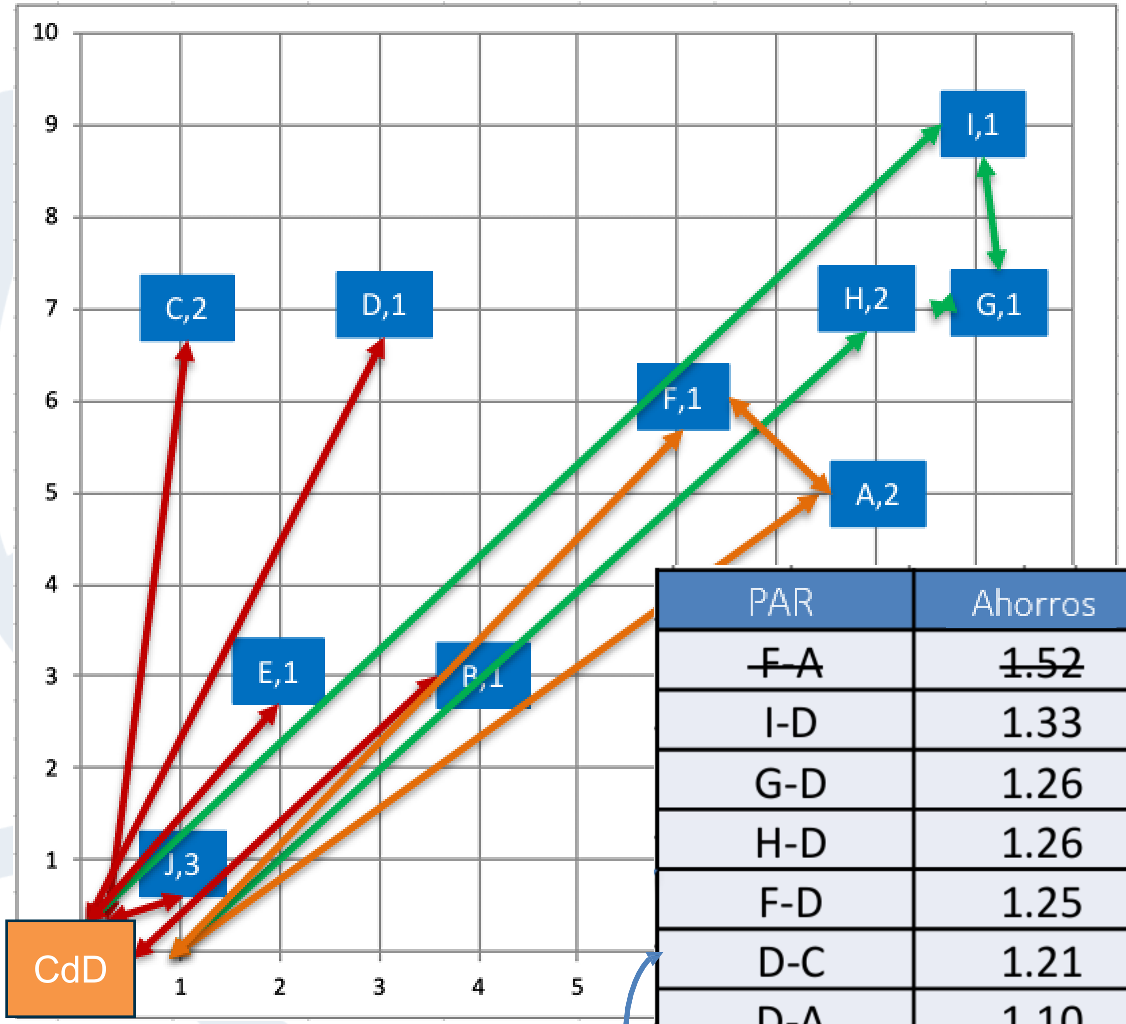
capacity = 1 bag + 2 bag = 3 ≤ 5 SI

time = (2.01 km)/(0.2 kpm) + 2(5 En) = 20.05 min. SI

Total distance = 11.30 - 1.52 = 9.78 km

Bidding solution

1. Calculate the savingsyo, $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$ of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under considerat $s_{i,j}$, include arc (i, j) in a path only if:
   - No ros $s_{i,j}$ e or vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.



Next 9 combinations

PAR	Ahorros
~~F-A~~	~~1.52~~
I-D	1.33
G-D	1.26
H-D	1.26
F-D	1.25
D-C	1.21
D-A	1.10
I-C	1.08
G-C	1.01

Current total distance of the route = 11.30 km

Consider joining F-A

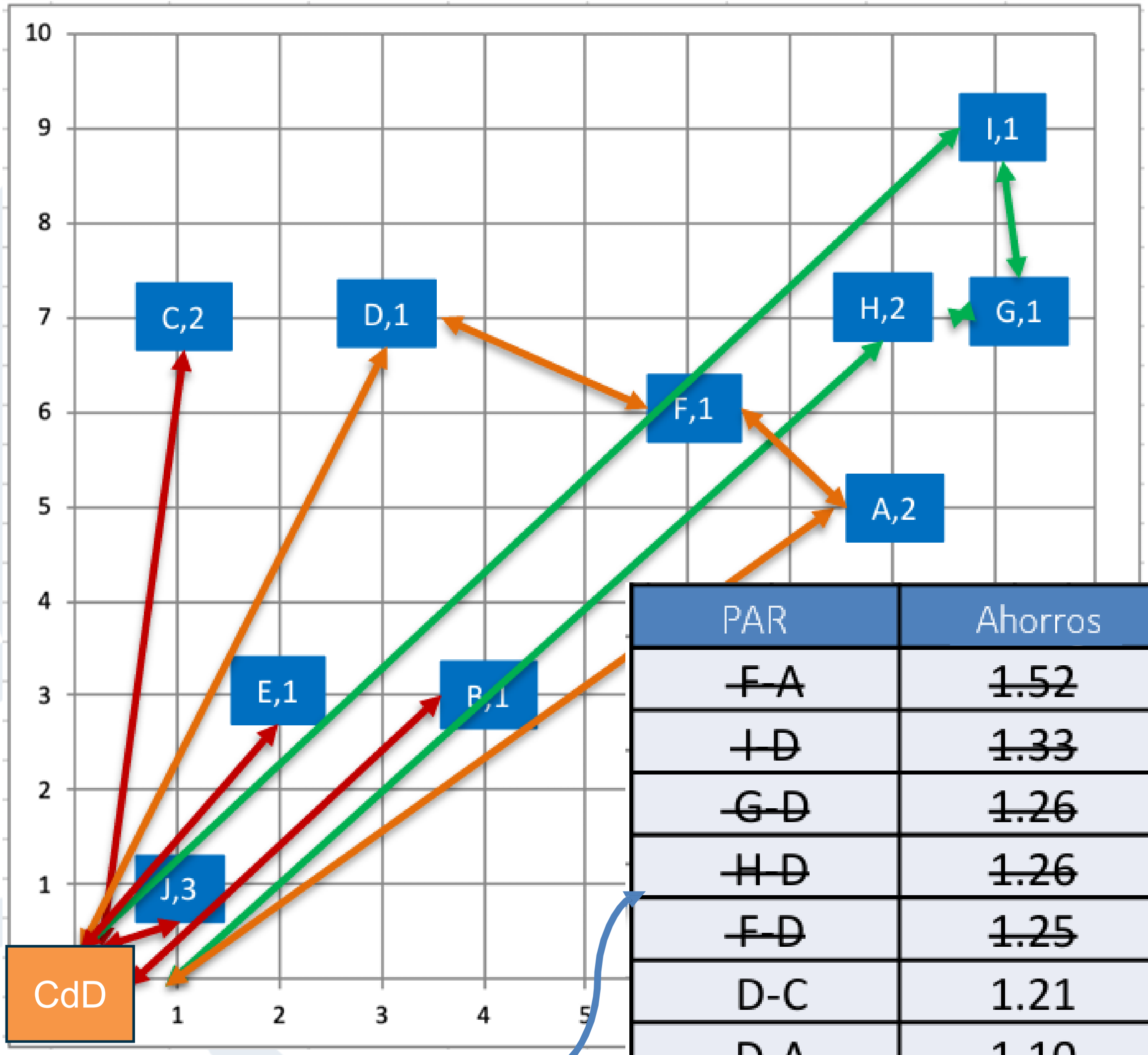
capacity = 1 bag + 2 bag = 3 ≤ 5 SI

time = (2.01 km)/(0.2 kpm) + 2(5 En) = 20.05 min. SI

Total distance = 11.30 - 1.52 = 9.78 km

Consider joining ID, GD, HD NO, > capability

Bidding solution



PAR	Ahorros
~~F-A~~	~~1.52~~
~~I-D~~	~~1.33~~
~~G-D~~	~~1.26~~
~~H-D~~	~~1.26~~
~~F-D~~	~~1.25~~
D-C	1.21
D-A	1.10
I-C	1.08
G-C	1.01

Next 9 combinations

1. Calculate the savings $s_{i,j} = c_{O,i} + c_{O,j} - c_{i,j}$ of demand nodes.
2. Sort and process savings in descending order of magnitude.
3. For the savings under consideration $s_{i,j}$, include arc (i, j) in a path only if:
   - No route or vehicle restrictions will be violated by adding it to a route, and
   - Nodes i and j are the first or last nodes to/from the origin on your current route.
4. If the savings list has not been exhausted, return to Step 3, processing the next entry in the list; otherwise, stop.

Current total distance of the route = 11.30 km

Consider joining F-A

capacity = 1 bag + 2 bag = 3 ≤ 5 SI

time = (2.01 km)/(0.2 kpm) + 2(5 En) = 20.05 min. SI

Total distance = 11.30 - 1.52 = 9.78 km

Consider joining ID, GD, HD NO, > capability

Consider joining FD

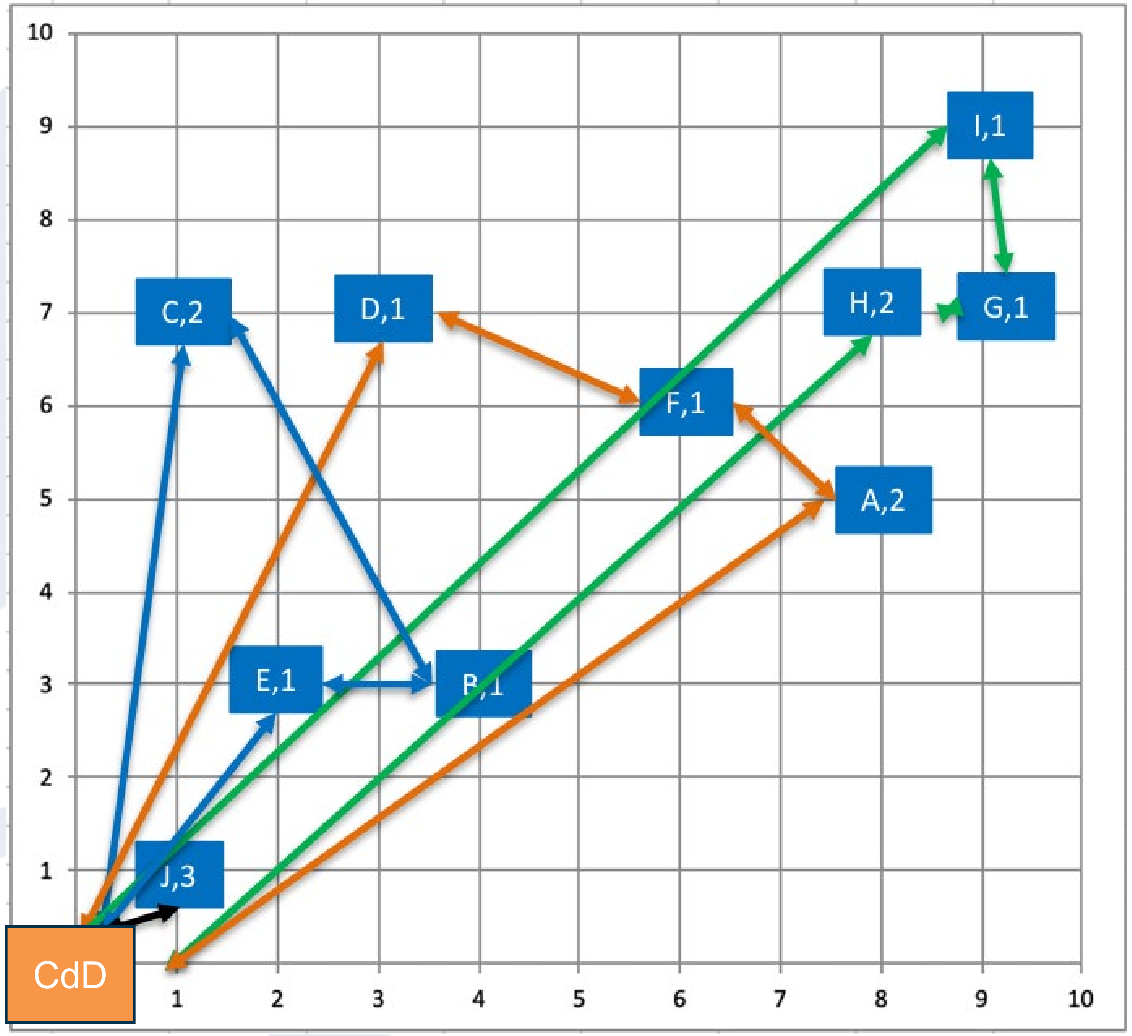
capacity = 1 bag + 3 bag = 4 ≤ 5SI

time = (2.24 km)/(0.2 kpm) + 3(5 En) = 26.20 min SI Total distance = 9.78 - 1.25 = 8.53 km

Consider joining DC, DA, IC, GC NO

And we would move on. . .

Bidding solution



Final solution:

4 Routes for a total of 6.92 km.

Green Route

DC-IGH-DC

4 bags, ~28 En. y 2.63 km

Orange Route

DC-DFA-DC

4 bags, ~26 En. y 2.24 km

Black Route

DC-J-DC

3 bags, ~6 En. y 0.28 km

Blue Route

DC-CBE-DC

4 bags, ~24 En. y 1.77 km

More about last mile solutions…

- Mathematical notes
  - The algorithm is fast, easy to implement and usually works well, but it is still a heuristic!
  - Can be modified to include additional restrictions (time and capacity)
  - More complicated requirements (e.g., time windows) are more difficult to implement and require more specialized techniques and algorithms (MILP can handle much of this).
- Other routing notes 1:∞.
  - Last mile routing becomes even more relevant with e-commerce
  - More and more companies are taking sustainability considerations (fuel, carbon, etc.) into account.
  - Newer approaches include outsourcing, crowdsourcing, different vehicles, etc.
  - You will probably NOT have to program your own routing algorithm, just understand how it works!

Note: In this example, we create actual routes with specific data. In many cases, we need to estimate the costs of future routes using forecasted or approximate data.

route-design-clarke-wright.ipynb

	Site	positionX	positionY	Demand	Description
0	DC	0	0	0	Distribution center
1	A	800	500	2	Consumer site
2	B	400	300	1	Consumer site
3	C	100	700	2	Consumer site
4	D	300	700	1	Consumer site
5	E	200	300	1	Consumer site
6	F	600	600	1	Consumer site
7	G	900	700	1	Consumer site
8	H	800	700	2	Consumer site
9	I	900	900	1	Consumer site
10	J	100	100	3	Consumer site

```
Found 5 routes
==============================================
Route 1: DC -> D -> A -> F -> DC | Distance: 2.37 km | Time: 26.86 minutes
Route 2: DC -> B -> C -> E -> DC | Distance: 1.77 km | Time: 23.86 minutes
Route 3: DC -> B -> C -> DC | Distance: 1.71 km | Time: 18.54 minutes
Route 4: DC -> H -> G -> I -> DC | Distance: 2.64 km | Time: 28.18 minutes
Route 5: DC -> J -> DC | Distance: 0.28 km | Time: 6.41 minutes
```